The physica package

Leedehai
GitHub | Typst

physica *noun.* Latin, study of nature.

Contents

1. Introduction 2
2. Using physica 2
3. The symbols 2
   3.1. Braces 2
   3.2. Vector notations 3
   3.3. Matrix notations 3
      3.3.1. Determinant, (anti-)diagonal, identity, zero matrix 3
      3.3.2. Jacobian matrix 4
      3.3.3. Hessian matrix 4
      3.3.4. Matrix with an element builder 5
      3.3.5. Rotation matrices, 2D and 3D 5
      3.3.6. Gram matrix 5
   3.4. Dirac braket notations 5
   3.5. Math text operators 6
   3.6. Differentials and derivatives 6
      3.6.1. Differentials 6
      3.6.2. Ordinary derivatives 7
      3.6.3. Partial derivatives (incl. mixed orders) 8
   3.7. Special show rules 9
      3.7.1. Matrix transpose with superscript T 9
      3.7.2. Matrix dagger with superscript + ... 10
   3.8. Miscellaneous 10
      3.8.1. Reduced Planck constant (hbar) 10
      3.8.2. Tensors 11
      3.8.3. Isotopes 12
      3.8.4. The n-th term in Taylor series 12
      3.8.5. Signal sequences (digital timing diagrams) 12
   3.9. Symbolic addition 13
4. Acknowledgement 15
5. License 15

1. Introduction

Typst is typesetting framework aiming to become the next generation alternative to LATEX. It excels in its friendly user experience and performance.

The `physica` package provides handy Typst typesetting functions that make academic writing for natural sciences simpler and faster, by simplifying otherwise very complex and repetitive expressions in the domain of natural sciences.

This manual itself was generated using the Typst CLI and the `physica` package, so hopefully this document is able to provide you with a sufficiently self evident demonstration of how this package shall be used.

2. Using `physica`

With `typst`'s package management:

```
#import "@preview/physica:0.9.8": *

$
curl(grad f), dd(x,y), pdv(,x,y,z,[2,k]), pdv(vb(e_i),x^j), \
tensor(Gamma,+k,-i,-j), braket(a, phi, b), imat(2, fill:*)
$
```

$$\nabla \times (\nabla f), \mathrm{d}x\,\mathrm{d}y, \frac{\partial^{k+3}}{\partial x^2 \partial y^k \partial z}, \frac{\partial \boldsymbol{e}_i}{\partial x^j},$$
$$\Gamma^k{}_{ij}, \langle a|\varphi|b\rangle, \begin{pmatrix} 1 & * \\ * & 1 \end{pmatrix}$$

3. The symbols

Some symbols are already provided as a Typst built-in. They are listed here just for completeness with annotation like typst `this`, as users coming from LATEX might not know they are already available in Typst out of box.

All symbols need to be used in **math mode** `$...$`.

3.1. Braces

Symbol	Abbr.	Example	Notes
typst `abs(`*expr*`)`		`abs(phi(x))` $\longrightarrow \lvert\varphi(x)\rvert$	absolute
typst `norm(`*expr*`)`		`norm(phi(x))` $\longrightarrow \lVert\varphi(x)\rVert$	norm
`Order(`*expr*`)`		`Order(x^2)` $\longrightarrow \mathcal{O}(x^2)$	big O
`order(`*expr*`)`		`order(1)` $\longrightarrow o(1)$	small O
`Set(`*expr* ; *condition*`)`		`Set(a_n), Set(a_i; forall i)` $\longrightarrow \{a_n\}, \{a_i \mid \forall i\}$ `Set(vec(1,n); forall n, n|2)` $\longrightarrow \left\{ \binom{1}{n} \;\middle\vert\; \forall n, n\vert 2 \right\}$	math set, use `Set` not `set` since the latter is a Typst keyword
`evaluated(`*expr*`)`		`evaluated(f(x))_0^oo` $\longrightarrow f(x)\vert_0^\infty$	attach a vertical bar on the right to denote evaluation boundaries

		`evaluated(f(x)/g(x))_0^1` $\longrightarrow \frac{f(x)}{g(x)}\Big\vert_0^1$	
`expectationvalue`	`expval`	`expval(u)` $\longrightarrow \langle u \rangle$ `expval(p,psi)` $\longrightarrow \langle \psi \vert p \vert \psi \rangle$	expectation value, also see braket Section 3.4 below

3.2. Vector notations

Symbol	Abbr.	Example	Notes
typst `vec(...)`		`vec(1,2)` $\longrightarrow \begin{pmatrix} 1 \\ 2 \end{pmatrix}$	column vector
`vecrow(...)`		`vecrow(alpha, b)` $\longrightarrow (\alpha, b)$ `vecrow(sum_0^n i, b, delim:"[")` $\longrightarrow \left[\sum_0^n i, b\right]$	row vector
`TT`		`v^TT, A^TT` $\longrightarrow v^{\mathsf{T}}, A^{\mathsf{T}}$	transpose, also see Section 3.7.1
`vectorbold(`*symbol*`)`	`vb`	`vb(a),vb(mu_1)` $\longrightarrow \boldsymbol{a}, \boldsymbol{\mu_1}$	vector, bold
`vectorunit(`*symbol*`)`	`vu`	`vu(a),vu(mu_1)` $\longrightarrow \hat{\boldsymbol{a}}, \hat{\boldsymbol{\mu}}_{\boldsymbol{1}}$	unit vector
`vectorarrow(`*symbol*`)`	`va`	`va(a),va(mu_1)` $\longrightarrow \vec{a}, \vec{\mu}_1$	vector, arrow (not bold: see ISO 80000-2:2019)
`grad`		`grad f` $\longrightarrow \boldsymbol{\nabla} f$	gradient
`div`		`div vb(E)` $\longrightarrow \boldsymbol{\nabla} \cdot \boldsymbol{E}$	divergence
`curl`		`curl vb(B)` $\longrightarrow \boldsymbol{\nabla} \times \boldsymbol{B}$	curl
`laplacian`		`diaer(u) = c^2 laplacian u` $\longrightarrow \ddot{u} = c^2 \nabla^2 u$	Laplacian, not typst `laplace` Δ
`dotproduct`	`dprod`	`a dprod b` $\longrightarrow a \cdot b$	dot product
`crossproduct`	`cprod`	`a cprod b` $\longrightarrow a \times b$	cross product
`innerproduct`	`iprod`	`iprod(u, v)` $\longrightarrow \langle u, v \rangle$ `iprod(sum_i a_i, b)` $\longrightarrow \left\langle \sum_i a_i, b \right\rangle$	inner product

3.3. Matrix notations

3.3.1. Determinant, (anti-)diagonal, identity, zero matrix

Symbol	Abbr.	Example	Notes
`TT`		`v^TT, A^TT` $\longrightarrow v^{\mathsf{T}}, A^{\mathsf{T}}$	transpose, also see Section 3.7.1
typst `mat(...)`		`mat(1,2;3,4)` $\longrightarrow \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$	matrix
`matrixdet(...)`	`mdet`	`mdet(1,x;1,y)` $\longrightarrow \begin{vmatrix} 1 & x \\ 1 & y \end{vmatrix}$	matrix determinant
`diagonalmatrix(...)`	`dmat`	`dmat(1,2)` $\longrightarrow \begin{pmatrix} 1 & \\ & 2 \end{pmatrix}$ `dmat(1,a,xi,delim:"[",fill:0)` $\longrightarrow \begin{bmatrix} 1 & 0 & 0 \\ 0 & a & 0 \\ 0 & 0 & \xi \end{bmatrix}$	diagonal matrix

`antidiagonalmatrix(...)`	`admat`	`admat(1,2)` $\longrightarrow \begin{pmatrix} & 1 \\ 2 & \end{pmatrix}$ `admat(1,a,xi,delim:"[",fill:dot)` $\longrightarrow \begin{bmatrix} \cdot & \cdot & 1 \\ \cdot & a & \cdot \\ \xi & \cdot & \cdot \end{bmatrix}$	anti-diagonal matrix
`identitymatrix(...)`	`imat`	`imat(2)` $\longrightarrow \begin{pmatrix} 1 & \\ & 1 \end{pmatrix}$ `imat(3,delim:"[",fill:*)` $\longrightarrow$ $\begin{bmatrix} 1 & * & * \\ * & 1 & * \\ * & * & 1 \end{bmatrix}$	identity matrix
`zeromatrix(...)`	`zmat`	`zmat(2)` $\longrightarrow \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix}$ `zmat(3,delim:"[")` $\longrightarrow$ $\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	zero matrix

3.3.2. Jacobian matrix

`jacobianmatrix(...)`, i.e. `jmat(...)`.

Typst (like LaTeX) cramps fractions in a matrix...

`jmat(f_1,f_2; x,y)`

$$\begin{pmatrix} \frac{\partial f_1}{\partial x} & \frac{\partial f_1}{\partial y} \\ \frac{\partial f_2}{\partial x} & \frac{\partial f_2}{\partial y} \end{pmatrix}$$

`jmat(f,g; x,y,z; delim:"[")`

$$\begin{bmatrix} \frac{\partial f}{\partial x} & \frac{\partial f}{\partial y} & \frac{\partial f}{\partial z} \\ \frac{\partial g}{\partial x} & \frac{\partial g}{\partial y} & \frac{\partial g}{\partial z} \end{bmatrix}$$

...but you can uncramp them using argument `big:#true` here

`jmat(f_1,f_2;x,y;big:#true)`

$$\begin{pmatrix} \dfrac{\partial f_1}{\partial x} & \dfrac{\partial f_1}{\partial y} \\ \dfrac{\partial f_2}{\partial x} & \dfrac{\partial f_2}{\partial y} \end{pmatrix}$$

`jmat(f,g;x,y,z;delim:"|",big:#true)`

$$\begin{vmatrix} \dfrac{\partial f}{\partial x} & \dfrac{\partial f}{\partial y} & \dfrac{\partial f}{\partial z} \\ \dfrac{\partial g}{\partial x} & \dfrac{\partial g}{\partial y} & \dfrac{\partial g}{\partial z} \end{vmatrix}$$

3.3.3. Hessian matrix

`hessianmatrix(...)`, i.e. `hmat(...)`.

Typst (like LaTeX) cramps fractions in a matrix...

`hmat(f; x,y)`

$$\begin{pmatrix} \frac{\partial^2 f}{\partial x^2} & \frac{\partial^2 f}{\partial x \partial y} \\ \frac{\partial^2 f}{\partial y \partial x} & \frac{\partial^2 f}{\partial y^2} \end{pmatrix}$$

`hmat(; x,y,z; delim:"[")`

$$\begin{bmatrix} \frac{\partial^2}{\partial x^2} & \frac{\partial^2}{\partial x \partial y} & \frac{\partial^2}{\partial x \partial z} \\ \frac{\partial^2}{\partial y \partial x} & \frac{\partial^2}{\partial y^2} & \frac{\partial^2}{\partial y \partial z} \\ \frac{\partial^2}{\partial z \partial x} & \frac{\partial^2}{\partial z \partial y} & \frac{\partial^2}{\partial z^2} \end{bmatrix}$$

...but you can uncramp them using argument `big:#true` here

`hmat(f;x,y;big:#true)`

$$\begin{pmatrix} \dfrac{\partial^2 f}{\partial x^2} & \dfrac{\partial^2 f}{\partial x \partial y} \\ \dfrac{\partial^2 f}{\partial y \partial x} & \dfrac{\partial^2 f}{\partial y^2} \end{pmatrix}$$

`hmat(;x,y,z;delim:"|",big:#true)`

$$\begin{vmatrix} \dfrac{\partial^2}{\partial x^2} & \dfrac{\partial^2}{\partial x \partial y} & \dfrac{\partial^2}{\partial x \partial z} \\ \dfrac{\partial^2}{\partial y \partial x} & \dfrac{\partial^2}{\partial y^2} & \dfrac{\partial^2}{\partial y \partial z} \\ \dfrac{\partial^2}{\partial z \partial x} & \dfrac{\partial^2}{\partial z \partial y} & \dfrac{\partial^2}{\partial z^2} \end{vmatrix}$$

3.3.4. Matrix with an element builder

`xmatrix(`*m, n, func*`)`, i.e. `xmat(...)`. The element building function *func* takes two integers which are the row and column numbers starting from 1.

```
#let g = (i,j) => $g^(#(i - 1)#(j - 1))$
xmat(2, 2, #g)
```

$$\begin{pmatrix} g^{00} & g^{01} \\ g^{10} & g^{11} \end{pmatrix}$$

3.3.5. Rotation matrices, 2D and 3D

`rot2mat(theta)`

$$\begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$$

`rot2mat(-a/2,delim:"[")`

$$\begin{bmatrix} \cos(-\frac{a}{2}) & -\sin(-\frac{a}{2}) \\ \sin(-\frac{a}{2}) & \cos(-\frac{a}{2}) \end{bmatrix}$$

`rot2mat(display(a/2),delim:"[")`

$$\begin{bmatrix} \cos\dfrac{a}{2} & -\sin\dfrac{a}{2} \\ \sin\dfrac{a}{2} & \cos\dfrac{a}{2} \end{bmatrix}$$

`rot3xmat(theta)`

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta \\ 0 & \sin\theta & \cos\theta \end{pmatrix}$$

`rot3ymat(45^degree)`

$$\begin{pmatrix} \cos 45^\circ & 0 & \sin 45^\circ \\ 0 & 1 & 0 \\ -\sin 45^\circ & 0 & \cos 45^\circ \end{pmatrix}$$

`rot3zmat(theta,delim:"[")`

$$\begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

3.3.6. Gram matrix

`grammat(alpha,beta)`

$$\begin{pmatrix} \langle\alpha,\alpha\rangle & \langle\alpha,\beta\rangle \\ \langle\beta,\alpha\rangle & \langle\beta,\beta\rangle \end{pmatrix}$$

`grammat(v_1,v_2,v_3, delim:"[")`

$$\begin{bmatrix} \langle v_1,v_1\rangle & \langle v_1,v_2\rangle & \langle v_1,v_3\rangle \\ \langle v_2,v_1\rangle & \langle v_2,v_2\rangle & \langle v_2,v_3\rangle \\ \langle v_3,v_1\rangle & \langle v_3,v_2\rangle & \langle v_3,v_3\rangle \end{bmatrix}$$

`grammat(v_1,v_2, norm:#true)`

$$\begin{pmatrix} \|v_1\|^2 & \langle v_1,v_2\rangle \\ \langle v_2,v_1\rangle & \|v_2\|^2 \end{pmatrix}$$

3.4. Dirac braket notations

Symbol	Abbr.	Example	Notes
`bra(`*content*`)`		`bra(u)` $\longrightarrow \langle u\|$ `bra(vec(1,2))` $\longrightarrow \left\langle \begin{pmatrix}1\\2\end{pmatrix} \right\|$	bra
`ket(`*content*`)`		`ket(u)` $\longrightarrow \|u\rangle$ `ket(vec(1,2))` $\longrightarrow \left\| \begin{pmatrix}1\\2\end{pmatrix} \right\rangle$	ket
`braket(..)`		`braket(a), braket(u, v)` $\longrightarrow \langle a\|a\rangle, \langle u\|v\rangle$ `braket(psi,A/N,phi)` $\longrightarrow \langle\psi\|\frac{A}{N}\|\varphi\rangle$	braket, with 1, 2, or 3 arguments
`ketbra(..)`		`ketbra(a), ketbra(u, v)` $\longrightarrow \|a\rangle\langle a\|, \|u\rangle\langle v\|$ `ketbra(a/N, b)` $\longrightarrow \|\frac{a}{N}\rangle\langle b\|$	ketbra, with 1 or 2 arguments
`expval(`*content*`)`		`expval(u)` $\longrightarrow \langle u\rangle$ `expval(A,psi)` $\longrightarrow \langle\psi\|A\|\psi\rangle$	expectation
`matrixelement(..)`	`mel`	`mel(n, partial_nu H, m)` $\longrightarrow \langle n\|\partial_\nu H\|m\rangle$	matrix element, same as `braket(n,M,n)`

Of course, expanded form is still supported if you put the equation in a single line, e.g.

```
$ bra(vec(1,2)), braket(psi, A/N, phi) $
```

$$\left\langle \begin{pmatrix} 1 \\ 2 \end{pmatrix} \right|, \left\langle \psi \middle| \frac{A}{N} \middle| \varphi \right\rangle$$

instead of `$bra(vec(1,2)), braket(psi, A/N, phi)$`, which leads to an inline form $\left\langle \begin{pmatrix} 1 \\ 2 \end{pmatrix} \right|, \langle \psi | \frac{A}{N} | \varphi \rangle$.

3.5. Math text operators

On top of Typst built-in math text operators (see "Predefined Operators" in `math.op`), this package adds the following:

`diag`, `rank`, `trace`, `Trace`, `Res`, `Re`, `Im`, `sgn`.

Also see show rule `super-T-as-transpose` (Section 3.7.1) to see how to add a matrix transpose operator more properly and smartly (A^{T}, not A^T).

3.6. Differentials and derivatives

Symbol	Abbr.	Example	Notes
`differential(...)`	`dd`	e.g. $\mathrm{d}f, \mathrm{d}x\,\mathrm{d}y, \mathrm{d}^3x, \mathrm{d}x \wedge \mathrm{d}y$ See Section 3.6.1	differential
`variation(...)`	`var`	`var(f)` $\longrightarrow \delta f$ `var(x,y)` $\longrightarrow \delta x\,\delta y$	variation, shorthand of `dd(..., d: delta)`
`difference(...)`		`difference(f)` $\longrightarrow \Delta f$ `difference(x,y)` $\longrightarrow \Delta x\,\Delta y$	difference, shorthand of `dd(..., d: Delta)`
`derivative(...)`	`dv`	e.g. $\frac{\mathrm{d}}{\mathrm{d}x}, \frac{\mathrm{d}f}{\mathrm{d}x}, \frac{\Delta^k f}{\Delta x^k}, \frac{\mathrm{d}f}{\mathrm{d}x}$ See Section 3.6.2	derivative
`partialderivative(...)`	`pdv`	e.g. $\frac{\partial}{\partial x}, \frac{\partial f}{\partial x}, \frac{\partial^4 f}{\partial x^2 \partial y^2}, \frac{\partial^5 f}{\partial x^2 \partial y^3}, \frac{\partial f}{\partial x}$ See Section 3.6.3	partial derivative, could be mixed order

3.6.1. Differentials

Functions: `differential(`**args*, ***kwargs*`)`, abbreviated as `dd(...)`.

- positional *args*: the variable names, **optionally** followed by an order number e.g. `2`, or an order array e.g. `[2,3]`, `[k]`, `[m n, lambda+1]`.
- named *kwargs*:
  - `d`: the differential symbol [default: `upright(d)`].
  - `prod`: the product symbol connecting the components [default: `none`].
  - `compact`: only effective if `p` is `none`. If `#true`, will remove the TeXBook-advised thin spaces between the d-units [default: `#false`].

TeXBook advises *[f]ormulas involving calculus look best when an extra thin space appears before dx or dy or d whatever* (Chapter 18 p.168), and this package heeds this advice. If you don't want the spaces between the d-units, you may pass a `compact:#true` argument: $\mathrm{d}r\,\mathrm{d}\theta$ vs. $\mathrm{d}r\mathrm{d}\theta$ (compact).

Order assignment algorithm:

- If there is no order number or order array, all variables have order 1.
- If there is an order number (not an array), then this order number is assigned to *every* variable, e.g. `dd(x,y,2)` assigns $x \leftarrow 2, y \leftarrow 2$.

- If there is an order array, then the orders therein are assigned to the variables in order, e.g. `dd(f,x,y,[2,3])` assigns $x \leftarrow 2, y \leftarrow 3$.
- If the order array holds fewer elements than the number of variables, then the orders of the remaining variables are 1, e.g. `dd(x,y,z,[2,3])` assigns $x \leftarrow 2, y \leftarrow 3, z \leftarrow 1$.
- If a variable x has order 1, it is rendered as $\mathrm{d}x$ not $\mathrm{d}^1 x$.

Examples

(1) `dd(f), r dd(r), f(r,theta) dd(r,theta)`

$$\mathrm{d}f, r\,\mathrm{d}r, f(r,\theta)\,\mathrm{d}r\,\mathrm{d}\theta$$

(2) `dd(x,3), dd(f,[k]), dd(f,[k],d:delta)`

$$\mathrm{d}^3 x, \mathrm{d}^k f, \delta^k f$$

(3) `dd(f,2), dd(vb(x),t,[3,])`

$$\mathrm{d}^2 f, \mathrm{d}^3 \boldsymbol{x}\,\mathrm{d}t$$

(4) `dd(x,y), dd(x,y,[2,3]), dd(x,y,z,[2,3])`

$$\mathrm{d}x\,\mathrm{d}y, \mathrm{d}^2 x\,\mathrm{d}^3 y, \mathrm{d}^2 x\,\mathrm{d}^3 y\,\mathrm{d}z$$

(5) `dd(x, y, z, [[1,1],rho+1,n_1])`

$$\mathrm{d}^{[1,1]} x\,\mathrm{d}^{\rho+1} y\,\mathrm{d}^{n_1} z$$

(6) `dd(x,y,d:Delta), dd(x,y,2,d:Delta)`

$$\Delta x\,\Delta y, \Delta^2 x\,\Delta^2 y$$

(7) `dd(t,x_1,x_2,x_3,prod:and)`

$$\mathrm{d}t \wedge \mathrm{d}x_1 \wedge \mathrm{d}x_2 \wedge \mathrm{d}x_3$$

(7) `dd(t,x_1,x_2,x_3,d:upright(D))`

$$\mathrm{D}t\,\mathrm{D}x_1\,\mathrm{D}x_2\,\mathrm{D}x_3$$

(8) `a dd(x) + b dd(y)`

$$a\,\mathrm{d}x + b\,\mathrm{d}y$$

3.6.2. Ordinary derivatives

Function: `derivative`(*f*, **args*, ***kwargs*), abbreviated as `dv(...)`.

- *f*: the function, which can be `#none` or omitted,
- positional *args*: the variable name, **optionally** followed by an order number e.g. 2,
- named *kwargs*:
  - `d`: the differential symbol [default: `upright(d)`].
  - `style`: the "slash" separating the numerator and denominator [default: `none`], by default it produces the normal fraction form $\frac{\mathrm{d}f}{\mathrm{d}x}$. The most common non-defaults are (1) `horizontal`, so as to create a flat form $\frac{\mathrm{d}f}{\mathrm{d}x}$ that fits inline; and (2) `large`, so that "d/dx" operator is put in front of the (potentially very large) function expression, and the size of the parentheses are adapted to match the highest of the operator and the function expression.

Order assignment algorithm: there is just one variable, so the assignment is trivial: simply assign the order number (default to 1) to the variable.

Examples

(1) `dv(,x), dv(,x,2), dv(f,x,k+1)`

$$\frac{\mathrm{d}}{\mathrm{d}x}, \frac{\mathrm{d}^2}{\mathrm{d}x^2}, \frac{\mathrm{d}^{k+1} f}{\mathrm{d}x^{k+1}}$$

(2) `dv(, vb(r)), dv(f, vb(r)_e, 2)`

$$\frac{\mathrm{d}}{\mathrm{d}\boldsymbol{r}}, \frac{\mathrm{d}^2}{\mathrm{d}\boldsymbol{r}_e^2}$$

(3) `dv((sum f_i (x) dd(x)),x,style:"large")`

$$\frac{\mathrm{d}}{\mathrm{d}x}\left(\sum f_i(x)\,\mathrm{d}x\right)$$

(4) `dv((u+v),t,2,d:upright(D),style:"large")`

$$\frac{\mathrm{D}^2}{\mathrm{D}t^2}\left(u+v\right)$$

(5) `dv(, x, d:delta), dv(, x, 2, d:Delta)`

(6) `dv(vb(u), t, 2, d: upright(D))`

$$\frac{\delta}{\delta x}, \frac{\Delta^2}{\Delta x^2}$$

(7) `dv(f,xi,k+1,style:"horizontal")`

$$\mathrm{d}^{k+1} f / \mathrm{d}\xi^{k+1}$$

$$\frac{\mathrm{D}^2 \boldsymbol{u}}{\mathrm{D}t^2}$$

(8)
`dv(vb(u),t,2,d:upright(D),style:"skewed")`

$$\mathrm{D}^2 \boldsymbol{u} \big/ \mathrm{D}t^2$$

3.6.3. Partial derivatives (incl. mixed orders)

Function: `partialderivative`(*f*, **args*, ***kwargs*), abbreviated as `pdv(...)`.

- *f*: the function, which can be `#none` or omitted,
- positional *args*: the variable names, **optionally** followed by an order number e.g. `2`, or an order array e.g. `[2,3]`, `[k]`, `[m n, lambda+1]`.
- named *kwargs*:
  - `d`: the differential symbol [default: `partial`].
  - `style`: the "slash" separating the numerator and denominator [default: `none`], by default it produces the normal fraction form $\frac{\partial f}{\partial x}$. The most common non-defaults non-defaults are (1) `horizontal`, so as to create a flat form $\frac{\mathrm{d}f}{\mathrm{d}x}$ that fits inline, and (2) `"large"`, so that "d/dx" operator is put in front of the (potentially very large) function expression, and the size of the parentheses are adapted to match the highest of the operator and the function expression.
  - `total`: the user-specified total order.
    - If it is absent, then (1) if the orders assigned to all variables are numeric, the total order number will be **automatically computed**; (2) if non-number symbols are present, computation will be attempted with minimum effort, and a user override with argument `total` might be necessary.

Order assignment algorithm:

- If there is no order number or order array, all variables have order 1.
- If there is an order number (not an array), then this order number is assigned to *every* variable, e.g. `pdv(f,x,y,2)` assigns $x \leftarrow 2, y \leftarrow 2$.
- If there is an order array, then the orders therein are assigned to the variables in order, e.g. `pdv(f,x,y,[2,3])` assigns $x \leftarrow 2, y \leftarrow 3$.
- If the order array holds fewer elements than the number of variables, then the orders of the remaining variables are 1, e.g. `pdv(f,x,y,z,[2,3])` assigns $x \leftarrow 2, y \leftarrow 3, z \leftarrow 1$.

Examples

(1) `pdv(,x), pdv(,t,2), pdv(,lambda,[k])`

$$\frac{\partial}{\partial x}, \frac{\partial^2}{\partial t^2}, \frac{\partial^k}{\partial \lambda^k}$$

(2) `pdv(phi,vb(r)), pdv(phi,vb(r)_e,2)`

$$\frac{\partial \varphi}{\partial \boldsymbol{r}}, \frac{\partial^2 \varphi}{\partial \boldsymbol{r}_e^2}$$

(3) `pdv(,x,y), pdv(,x,y,2)`

$$\frac{\partial^2}{\partial x \partial y}, \frac{\partial^4}{\partial x^2 \partial y^2}$$

(4) `pdv(f,x,y,2), pdv(f,x,y,3)`

$$\frac{\partial^4 \varphi}{\partial x^2 \partial y^2}, \frac{\partial^6 \varphi}{\partial x^3 \partial y^3}$$

(5) `pdv(,x,y,[2,]), pdv(,x,y,[1,2])`

$$\frac{\partial^3}{\partial x^2 \partial y}, \frac{\partial^3}{\partial x \partial y^2}$$

(6) `pdv(,z)[integral_0^x f(x,y) dd(x,y)]`

$$\frac{\partial}{\partial z} \left[\int_0^z f(x) \, \mathrm{d}x \, \mathrm{d}y \right]$$

(7) `pdv(,y)[pdv((x+y),x,style:"large")]`

(8) `pdv(f,x,y,style:"horizontal")`

$$\frac{\partial}{\partial y}\left[\frac{\partial}{\partial x}\left(x+y\right)\right]$$

$$\partial^2 f/\partial x\partial y$$

(9) `pdv(, (x^1), (x^2), (x^3), [1,3])`

$$\frac{\partial^5}{\partial(x^1)\partial(x^2)^3\partial(x^3)}$$

(10) `pdv(phi,x,y,z,tau, [2,2,2,1])`

$$\frac{\partial^7\varphi}{\partial x^2\partial y^2\partial z^2\partial\tau}$$

(11) `pdv(,x,y,z,t,[1,xi,2,eta+2])`

$$\frac{\partial^{\eta+\xi+5}}{\partial x\partial y^{\xi}\partial z^2\partial t^{\eta+2}}$$

(12) `pdv(,x,y,z,[xi n,n-1],total:(xi+1)n)`

$$\frac{\partial^{(\xi+1)n}}{\partial x^{\xi n}\partial y^{n-1}\partial z}$$

(13) `pdv(S, phi.alt, phi, d:delta)`

$$\frac{\delta^2 S}{\delta\phi\delta\varphi}$$

(14) `pdv(W[J], J^mu (x), J^nu (y), d:delta)`

$$\frac{\delta^2 W[J]}{\delta J^{\mu}(x)\delta J^{\nu}(y)}$$

(15) `integral_V dd(V)(pdv(cal(L), phi) - partial_mu (pdv(cal(L), (partial_mu phi)))) = 0`

$$\int_V \mathrm{d}V\left(\frac{\partial\mathcal{L}}{\partial\varphi}-\partial_\mu\left(\frac{\partial\mathcal{L}}{\partial(\partial_\mu\varphi)}\right)\right)=0$$

3.7. Special show rules

3.7.1. Matrix transpose with superscript T

Matrix transposition can be simply written as `..^T`, just like handwriting, and the only superscript `T` will be formatted properly to represent transposition instead of a normal capital letter T.

This $\square^T \Rightarrow \square^{\mathsf{T}}$ conversion is disabled if the base is either

- a `limits(...)` or `scripts(...)` element, or
- an integration $\int$ or sum $\sum$ (not greek Σ) or product $\prod$ (not greek Π) or vertical bar $|$, or
- an equation or `lr(...)` element whose last child is one of the above.

Overrides: if you really want to

- print a transpose explicitly: use symbol `TT`: `A^TT` $\Rightarrow A^{\mathsf{T}}$;
- print a superscript letter T: use `scripts(T)`: `2^scripts(T)` $\Rightarrow 2^T$.

This feature needs to be enabled explicitly through a *show rule*.

```
#show: super-T-as-transpose
(A B)^T = B^T A^T
```

If you only want to enable it within a content block's scope, you may do

```
#[
  #show: super-T-as-transpose // Enabled from here till the end of block.
  (A B)^T = B^T A^T
]
```

Examples

(1) `(U V_n W')^T = W'^T V_n^T U^T`

$$(\Sigma V_n W')^{\mathsf{T}} = W^{\mathsf{T}} V_n^{\mathsf{T}} \Sigma^{\mathsf{T}}$$

(2) `vec(a, b)^T, mat(a, b; c, d)^T`

$$\begin{pmatrix} a \\ b \end{pmatrix}^{\mathsf{T}}, \begin{pmatrix} a & b \\ c & d \end{pmatrix}^{\mathsf{T}}$$

(3) `abs(a)^T, norm(a)^T, evaluated(F(t))^T_0`

$$|a|^T, \|a\|^T, F(t)|_0^T$$

(4) `integral^T, sum^T, product^T`

$$\int^T, \sum^T, \prod^T$$

(5) `limits(e)^T, scripts(e)^T`

$$\overset{T}{e}, e^T$$

(6) `(M+N)^T, (m+n)^scripts(T)`

$$(M+N)^{\mathsf{T}}, (m+n)^T$$

3.7.2. Matrix dagger with superscript +

The conjugate transpose, also known as the Hermitian transpose, transjugate, or adjoint, of a complex matrix A is performed by first transposing and then complex-conjugating each matrix element. It is often denoted as A^*, A^{H}, and sometimes with a dagger symbol: `A^dagger` $\Rightarrow A^\dagger$.

Writing `..^dagger` often visually clutters an equation in the source code form. Therefore, the package offers the ability to write `..^+` instead.

This $\square^+ \Rightarrow \square^\dagger$ conversion is disabled if the base is either
- a `limits(...)` or `scripts(...)` element, or
- an equation or `lr(...)` element whose last child is one of the above.

This feature needs to be enabled explicitly through a *show rule*.

```
#show: super-plus-as-dagger
U^+U = U U^+ = I
```

If you only want to enable it within a content block's scope (e.g. you want to have $\square^+$ for ions or Moore–Penrose inverse outside the block), you may do

```
#[
  #show: super-plus-as-dagger // Enabled from here till the end of block.
  U^+U = U U^+ = I
]
```

Overrides: if you really want to
- print a dagger explicitly: use the built-in symbol `dagger` as normal: `A^dagger` $\Rightarrow A^\dagger$;
- print a superscript plus sign: use `scripts(+)`: `A^scripts(+)` $\Rightarrow A^+$.

Examples

(1) `U^+U = U U^+ = I`

$$U^\dagger U = UU^\dagger = I$$

(2) `mat(1+i,1;2-i,1)^+ = mat(1-i,2+i;1,1)`

$$\begin{pmatrix} 1+i & 1 \\ 2-i & 1 \end{pmatrix}^\dagger = \begin{pmatrix} 1-i & 2+i \\ 1 & 1 \end{pmatrix}$$

(3) `limits(N)^+, scripts(N)^+`

$$\overset{+}{N}, N^+$$

(4) `#let eq = $scripts(N)$; eq^+`

$$N^+$$

3.8. Miscellaneous

3.8.1. Reduced Planck constant (hbar)

In the default font, the Typst built-in symbol `planck` $\hbar$ looks a bit off: on letter "h" there is a slash instead of a horizontal bar, contrary to the symbol's colloquial name "h-bar". This package offers `hbar` to render the symbol in the familiar form: $\hbar$. Contrast:

Typst's `planck`	$E = \hbar\omega$	$\frac{\pi G^2}{\hbar c^4}$	$Ae^{\frac{i(px-Et)}{\hbar}}$	$i\hbar\frac{\partial}{\partial t}\psi = -\frac{\hbar^2}{2m}\nabla^2\psi$
this package's `hbar`	$E = \hbar\omega$	$\frac{\pi G^2}{\hbar c^4}$	$Ae^{\frac{i(px-Et)}{\hbar}}$	$i\hbar\frac{\partial}{\partial t}\psi = -\frac{\hbar^2}{2m}\nabla^2\psi$

Known limitation: `hbar` uses the `strike` function, and show rules of `strike` will affect `hbar`. Therefore, you may have to revert your show rules for `hbar`. The following is an example.

```
#import "@preview/physica:0.9.8": hbar as old-hbar

#show strike: set text(gray)
#let hbar = {
  show strike: set text(black)
  old-hbar
}

$hbar$ is black, while $#old-hbar$ is gray.
```

3.8.2. Tensors

Tensors are often expressed using the abstract index notation, which makes the contravariant and covariant "slots" explicit. The intuitive solution of using superscripts and subscripts do not suffice if both upper (contravariant) and lower (covariant) indices exist, because the notation rules require the indices be vertically separated: e.g. $T^a{}_b$ and $T_a{}^b$, which are of different shapes. "T^a_b" is flatly wrong, and `T^(space w)_(i space j)` produces a weird-looking "$T_i{}^w{}_j$" (note w, j vertically overlap).

Function: `tensor(`*symbol*, **args*`)`.

- *symbol*: the tensor symbol,
- positional *args*: each argument takes the form of $+\ldots$ or $-\ldots$, where a `+` prefix denotes an upper index and a `-` prefix denotes a lower index.

Examples

(1) `tensor(u,+a), tensor(v,-a)`

$$u^a, v_a$$

(2) `tensor(h,+mu,+nu), tensor(g,-mu,-nu)`

$$h^{\mu\nu}, g_{\mu\nu}$$

(3) `tensor(T,+a,-b), tensor(T,-a,+b)`

$$T^a{}_b, T_a{}^b$$

(4) `tensor(T, -i, +w, -j)`

$$T_i{}^w{}_j$$

(5) `tensor((dd(x^lambda)),-a)`

$$\left(\mathrm{d}x^\lambda\right)_a$$

(6) `tensor(AA,+a,+b,-c,-d,+e,-f,+g,-h)`

$$\mathbb{A}^{ab}{}_{cd}{}^{e}{}_{f}{}^{g}{}_{h}$$

(7) `tensor(R, -a, -b, +d)`

$$R_{ab}{}^d$$

(8) `tensor(T,+1,-I(1,-1),+a_bot,-+,+-)`

$$T^1{}_{I(1,-1)}{}^{a_\perp}{}_{+}{}^{-}$$

(9) `grad_mu A^nu = partial_mu A^nu + tensor(Gamma,+nu,-mu,-lambda) A^lambda`

$$\nabla_\mu A^\nu = \partial_\mu A^\nu + \Gamma^\nu{}_{\mu\lambda} A^\lambda$$

(Though for those of you who are studying for math exams, I'd like to remind you Christoffel symbol Γ is technically not a tensor!)

3.8.3. Isotopes

Function: `isotope(`*element*, *a*: ..., *z*: ...`)`.
- *element*: the chemical element (use `"..."` for multi-letter symbols)
- *a*: the mass number A [default: `none`].
- *z*: the atomic number Z [default: `none`].

Examples

(1) `isotope(I, a:127)`

$$^{127}\mathrm{I}$$

(2) `isotope("Fe", z:26)`

$$_{26}\mathrm{Fe}$$

(3) `isotope("Bi",a:211,z:83) -> isotope("Tl",a:207,z:81) + isotope("He",a:4,z:2)`

$$^{211}_{83}\mathrm{Bi} \to {}^{207}_{81}\mathrm{Tl} + {}^{4}_{2}\mathrm{He}$$

(4) `isotope("Tl",a:207,z:81) -> isotope("Pb",a:207,z:82) + isotope(e,a:0,z:-1)`

$$^{207}_{81}\mathrm{Tl} \to {}^{207}_{82}\mathrm{Pb} + {}^{0}_{-1}\mathrm{e}$$

3.8.4. The n-th term in Taylor series

Function: `taylorterm(`*func*, *x*, *x0*, *idx*`)`.
- *func*: the function e.g. `f`, `(f+g)`,
- *x*: the variable name e.g. `x`,
- *x0*: the variable value at the expansion point e.g. `x_0`, `(1+a)`,
- *idx*: the index of the term, e.g. `0`, `1`, `2`, `n`, `(n+1)`.

If *x0* or *idx* is an add/sub sequence e.g. `-a`, `a+b`, then the function automatically adds a parenthesis where appropriate.

Examples

(1) `taylorterm(f,x,x_0,0)`

$$f(x_0)$$

(2) `taylorterm(f,x,x_0,1)`

$$f^{(1)}(x_0)(x - x_0)$$

(3) `taylorterm(F,x^nu,x^nu_0,n)`

$$\frac{F^{(n)}(x_0^\nu)}{n!}(x^\nu - x_0^\nu)^n$$

(4) `taylorterm(f,x,x_0,n)`

$$\frac{f^{(n)}(x_0)}{n!}(x - x_0)^n$$

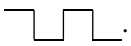
(5) `taylorterm(f,x,1+a,2)`

$$\frac{f^{(2)}(1+a)}{2!}(x - (1+a))^2$$

(6) `taylorterm(f_p,x,x_0,n-1)`

$$\frac{f_p^{(n-1)}(x_0)}{(n-1)!}(x - x_0)^{n-1}$$

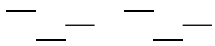
3.8.5. Signal sequences (digital timing diagrams)

In engineering, people often need to draw digital timing diagrams for signals, like  .

Function: `signals(str, step::..., style:...)`.
- `str`: a string representing the signals. Each character represents an glyph (see below).
- `step` (optional): step width, i.e. how wide each glyph is [default: `#1em`].
- `color` (optional): the stroke color [default: `#black`].

Glyph characters

`HLM` ⟺ `"10-"` full step



`hlm ^v` 1/2 step, 1/10 step

`| ' ,` (edge) 0 step

`= #` empty, shaded

`R` (rise)

`F` (fall)

`C` (charge)

`D` (drain)

`<`

`>`

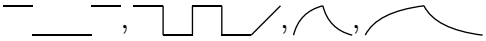
`X`

ignore: (blankspace)
separate: `&`

repeat: `.` (dot)

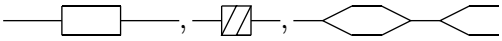
Examples

(1) `signals("10.1"), signals("1|0|1|0R"), signals("CD"), signals("CD", step: #2em)`

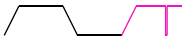


(2) `signals("M'H|L|h|l|^|v,&|H'M'H|l,m,l|")` (the ampersand & serves as a separator)

(3) `signals("-|=|-", step: #2em), signals("-|#|-"), signals("-<=>-<=")`

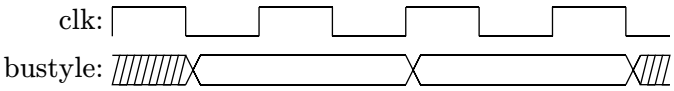


(4) `signals("R1..F0..", step: #.5em)signals("R1.|v|1", step: #.5em, color:#fuchsia)`



(5)

```
"clk:" & signals("|1....|0....|1....|0....|1....|0....|1....|0..", step: #0.5em) \
"bustyle:" & signals(" #.... X=... ..... ..... X=... ..... ..... X#.", step: #0.5em)
```



3.9. Symbolic addition

This package implements a very rudimentary, **bare-minimum-effort** symbolic addition function to aid the automatic computation of a partial derivative's total order in the absence of user override (see Section 3.6.3). Though rudimentary and unsophisticated, this should suffice for most use cases in partial derivatives.

Function: `BMEsymadd([...])`.

- `...`: symbols that need to be added up e.g. `[1,2]`, `[a+1,b^2+1,2]`.

Examples

(1) `BMEsymadd([1]), BMEsymadd([2, 3])` $\longrightarrow$ $1, 5$

(2) `BMEsymadd([a, b^2, 1])` $\longrightarrow$ $a + b^2 + 1$

(3) `BMEsymadd([a+1,2c,b,2,b])` $\longrightarrow$ $a + 2b + 2c + 3$

(4) `BMEsymadd([a+1,2(b+1),1,b+1,15])` $\longrightarrow$ $a + b + 2(b + 1) + 18$

(5) `BMEsymadd([a+1,2(b+1),1,(b+1),15])` $\longrightarrow$ $a + 3(b + 1) + 17$

(6) `BMEsymadd([a+1,2(b+1),1,3(b+1),15])` $\longrightarrow$ $a + 5(b + 1) + 17$

(7) `BMEsymadd([2a+1,xi,b+1,a xi + 2b+a,2b+1])` $\longrightarrow$ $3a + 5b + \xi + a\xi + 3$

4. Acknowledgement

Huge thanks to these LATEX packages, for lighting the way of physics typesetting.

- `physics` by Sergio C. de la Barrera,
- `derivatives` by Simon Jensen,
- `tensor` by Philip G. Ratcliffe et al.

5. License

Source code: © Copyright 2023 Leedehai. See the license here.

The document: Creative Commons Attribution-NoDerivatives 4.0 license (CC BY-ND 4.0).